



# Catálogo de Questões Teóricas e Práticas de Exame — PP

## Formato de Resposta Exigido pelos Docentes:

1. **Direto e Conciso:** Definição clara sem texto supérfluo.
2. **Pequeno Exemplo em Código Java:** Exemplo mínimo, funcional e demonstrativo.
3. **0% `java.util`:** Uso estrito de arrays nativos Java ( `[]` ) e contadores manuais.



## Pergunta 1: Classes Abstratas vs Interfaces

**Tema:** Abstração e Modelação Orientada a Objetos.

### Pergunta:

Explique as diferenças entre classes abstratas e interfaces em Java. Em que situações é mais adequado optar por cada uma? Ilustre com um exemplo prático.

### Resposta Direta:

- **Classe Abstrata:** É uma superclasse parcial que pode conter **estado (atributos)**, construtores, métodos concretos e métodos abstratos ( `abstract` ). Permite herança simples ( `extends` ). Deve ser usada quando existe uma relação forte de parentesco ("É UM") e partilha de código ou estado entre classes correlacionadas.
- **Interface:** É um **contrato puro de comportamento** que define *o que* a classe faz, sem guardar estado de instância (apenas constantes `public static final` ) nem construtores. Uma classe pode implementar múltiplas interfaces ( `implements A, B` ). Deve ser usada para definir capacidades comuns a classes que não partilham a mesma hierarquia.

### Exemplo em Código Java (Sem `java.util`):

```
// Classe Abstrata: Partilha estado e construtor
public abstract class Veiculo {
    private String codigo;
    public Veiculo(String codigo) { this.codigo = codigo; }
    public String getCodigo() { return this.codigo; }
    public abstract double calcularAutonomia(); // Método abstrato
}

// Interface: Contrato de comportamento puro
```

```
public interface Refrigeravel {
    double getTemperaturaAlvo();
}

// Subclasse: Herda da classe abstrata e implementa a interface
public class VeiculoRefrigerado extends Veiculo implements Refrigeravel {
    private double temp;
    public VeiculoRefrigerado(String codigo, double temp) {
        super(codigo);
        this.temp = temp;
    }
    @Override
    public double calcularAutonomia() { return 500.0; }
    @Override
    public double getTemperaturaAlvo() { return this.temp; }
}
```

## ✚ Pergunta 2: Passagem de Parâmetros em Java

**Tema:** Semântica da Linguagem e Gestão de Memória.

### Pergunta:

Descreva como Java realiza a passagem de argumentos para os métodos, distinguindo tipos primitivos de referências a objetos. Ilustre com um exemplo concreto dos seus efeitos.

### Resposta Direta:

Java realiza a passagem de parâmetros **EXCLUSIVAMENTE POR VALOR**:

1. **Tipos Primitivos:** É passada uma cópia do próprio valor. Alterações à variável no método **não afetam** o valor original fora do método.
2. **Referências a Objetos:** É passada uma cópia do endereço de memória (referência) do objeto na Heap.
  - Modificar os atributos internos do objeto apontado pela referência **altera o objeto original**.
  - Reatribuir a variável de referência para apontar para um novo objeto ( `obj = new ...` ) **não altera a referência original** fora do método.

### Exemplo em Código Java (Sem `java.util`):

```
public class TestePassagem {
    public static void alterarPrimitivo(int x) {
        x = 99; // Nao afeta o original
    }

    public static void alterarObjeto(AidBox box) {
        box.setCodigo("BOX_NOVA"); // Afeta o objeto original na Heap!
        box = new AidBox("OUTRA"); // Nao altera a referencia original fora do metodo
    }
}
```

```

public static void main(String[] args) {
    int val = 10;
    alterarPrimitivo(val); // val continua 10

    AidBox minhaBox = new AidBox("BOX_ORIGINAL");
    alterarObjeto(minhaBox); // minhaBox passa a ter o código "BOX_NOVA"
}
}

```

### ✖ Pergunta 3: Conversão de Tipos (*Casting*), `instanceof` e `ClassCastException`

**Tema:** Polimorfismo e Tipagem Dinâmica.

#### Pergunta:

Explique o conceito de casting no contexto da herança e polimorfismo, distinguindo Upcasting de Downcasting e como prevenir exceções em tempo de execução.

#### Resposta Direta:

- **Upcasting:** Conversão de uma subclasse para um tipo de superclasse ou interface. É **automática e sempre segura**, pois qualquer subclasse é uma instância da sua superclasse.
- **Downcasting:** Conversão explícita de uma superclasse para um tipo de subclasse. É **potencialmente perigosa** porque o objeto real na Heap pode não ser do tipo da subclasse destino. Se o tipo não for compatível, a JVM lança uma `ClassCastException`.
- **Prevenção:** Deve utilizar-se o operador `instanceof` antes de efetuar *downcasting*.

#### Exemplo em Código Java (Sem `java.util`):

```

Veiculo v = new VeiculoRefrigerado("V01", 1000, 200.0); // Upcasting automatico

// Downcasting Seguro com instanceof
if (v instanceof VeiculoRefrigerado) {
    VeiculoRefrigerado vr = (VeiculoRefrigerado) v; // Downcasting explícito seguro
    System.out.println("Km Max: " + vr.getMaxQuilometros());
} else {
    System.out.println("O veiculo nao e refrigerado!");
}

```

### ✖ Pergunta 4: Identidade ( `==` ) vs Igualdade ( `equals` ) e `toString()`

**Tema:** Contratos da Classe `java.lang.Object`.

## Pergunta:

Distinga a igualdade de identidade ( `==` ) da igualdade lógica ( `equals()` ). Mostre como redefinir corretamente os métodos `equals()` e `toString()` .

## Resposta Direta:

- **Operador `==`** : Compara a **identidade de referências**, verificando se duas variáveis apontam para o exato mesmo endereço de memória na Heap. Em primitivos, compara os valores diretamente.
- **Método `equals()`** : Compara a **igualdade lógica**, verificando se os atributos dos objetos possuem os mesmos valores. Por defeito em `Object` , o `equals()` comporta-se como `==` , devendo ser redefinido na classe.
- **Método `toString()`** : Devolve uma representação textual e legível do estado do objeto.

## Exemplo em Código Java (Sem `java.util`):

```
public class ContainerImpl implements Container {
    private String id;
    private double capacity;

    public ContainerImpl(String id, double capacity) {
        this.id = id;
        this.capacity = capacity;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Mesmo endereço
        if (obj == null || !(obj instanceof Container)) return false; // Nulo ou outro tipo

        Container other = (Container) obj;
        if (this.id == null) return other.getId() == null;
        return this.id.equals(other.getId()); // Comparacao logica pelo ID
    }

    @Override
    public String toString() {
        return "Container[ID=" + this.id + ", Capacidade=" + this.capacity + "];"
    }
}
```

## Pergunta 5: Exceções Verificadas (*Checked*) vs Não Verificadas (*Unchecked*) e `try-catch-finally`

**Tema:** Tratamento de Erros e Robustez de Código.

## Pergunta:

Distinga Checked Exceptions de Unchecked Exceptions. Explique a ordem de execução do bloco `try-catch-finally` e a utilidade do bloco `finally`.

## Resposta Direta:

- **Checked Exceptions (Verificadas):** Subclasses de `Exception` (exceto `RuntimeException`). O compilador **obriga** o seu tratamento ( `try-catch` ) ou declaração na assinatura do método ( `throws` ). Representam condições de erro externas e recuperáveis (ex: falhas de I/O, ficheiros inexistentes).
- **Unchecked Exceptions (Não Verificadas):** Subclasses de `RuntimeException` (ex: `NullPointerException`, `ArrayIndexOutOfBoundsException`). O compilador não exige o seu tratamento. Representam erros de lógica e programação.
- **Bloco `try-catch-finally`:** O código do `try` é executado. Se houver exceção, o fluxo salta para o `catch` correspondente. O bloco `finally` **é executado SEMPRE**, haja ou não exceção ou instruções `return`, sendo usado para libertar recursos.

## Exemplo em Código Java (Sem `java.util`):

```
public class ExemploExcecoes {
    public static void adicionarAidBox(Route route, AidBox box) throws RouteException {
        if (box == null) {
            throw new RouteException("AidBox invalida (nula!)"); // Checked Exception
        }
        route.addAidBox(box);
    }

    public static void main(String[] args) {
        try {
            adicionarAidBox(null, null);
        } catch (RouteException e) {
            System.out.println("Erro capturado: " + e.getMessage());
        } finally {
            System.out.println("Bloco finally executado obrigatoriamente.");
        }
    }
}
```

---

## Pergunta 6: Modificadores de Acesso e Encapsulamento

---

**Tema:** Proteção de Dados e Modelação.

## Pergunta:

Ordene os modificadores de acesso em Java do mais restritivo para o mais permissivo e explique o princípio de encapsulamento.

### Resposta Direta:

- **Princípio de Encapsulamento:** Consiste em ocultar o estado interno de um objeto e forçar todo o acesso a ser realizado através de métodos públicos controlados (*getters/setters*), garantindo a integridade dos dados.
- **Modificadores de Acesso (da menor visibilidade para a maior):**
  1. `private` : Apenas visível na própria classe.
  2. (*default* / *package-private*): Visível no mesmo pacote (*package*).
  3. `protected` : Visível no mesmo pacote e por subclasses em qualquer pacote.
  4. `public` : Visível em qualquer classe do projeto.

## Pergunta 7: Sobrecarga (Overloading) vs Sobreposição (Overriding)

**Tema:** Polimorfismo Estático vs Dinâmico.

### Pergunta:

Diferencie Sobrecarga de Sobreposição de métodos, indicando quando é decidida a execução de cada uma.

### Resposta Direta:

- **Sobrecarga (Overloading) — Polimorfismo Estático:** Ocorre na mesma classe com métodos de igual nome mas parâmetros diferentes. A decisão de qual método chamar é feita pelo compilador em **tempo de compilação** (*compile-time*).
- **Sobreposição (Overriding) — Polimorfismo Dinâmico:** Ocorre em herança/interfaces quando uma subclasse redefine um método herdado mantendo a mesma assinatura. Usa-se `@Override` . A decisão é feita em **tempo de execução** (*runtime*) com base no objeto real alocado na Heap.

## Pergunta 8: Membros Estáticos ( `static` ) e a Palavra-Chave `final`

**Tema:** Modificadores de Escopo e Imutabilidade.

### Pergunta:

Explique o significado do modificador `static` e o efeito da palavra reservada `final` em variáveis,

métodos e classes.

### Resposta Direta:

- **static** : Define que o membro pertence à **classe** e não às instâncias. Existe apenas 1 cópia em memória. Métodos estáticos não podem usar `this` , `super` ou aceder a atributos de instância.
- **final** :
  - **Em Variáveis:** Torna a variável constante (não pode ser reatribuída). Se for um objeto, a referência não muda, mas o estado interno do objeto pode mudar.
  - **Em Métodos:** Impede que o método seja sobreposto (*overridden*) nas subclasses.
  - **Em Classes:** Impede que a classe seja herdada/estendida.

## Pergunta 9: Composição vs Herança

**Tema:** Padrões de Reutilização de Código.

### Pergunta:

Compare Composição ("Tem Um") e Herança ("É Um"). Por que razão é recomendado preferir composição?

### Resposta Direta:

- **Herança ("É UM"):** A subclasse herda estado e comportamento da superclasse. Cria um **acoplamento forte**; alterações na superclasse podem quebrar as subclasses (problema da superclasse frágil).
- **Composição ("TEM UM"):** Uma classe inclui instâncias de outras classes como atributos e delega-lhes tarefas. Promove o **acoplamento fraco**, encapsulamento total e permite alterar o comportamento em tempo de execução.

## Pergunta 10: Enums com Estado e Métodos

**Tema:** Tipos Enumerados Avançados.

### Pergunta:

Demonstre como declarar um `enum` em Java com atributos de estado, construtor e métodos.

### Resposta Direta:

Os `enum` em Java são tipos estruturados cujos construtores são obrigatoriamente **privados**. Podem conter atributos `final` e métodos.

```
public enum ContainerType {
    PERISHABLE_FOOD("Comida Perecível", true),
    CLOTHING("Vestuario", false);

    private final String descricao;
    private final boolean requerRefrigeracao;

    private ContainerType(String descricao, boolean requerRefrigeracao) {
        this.descricao = descricao;
        this.requerRefrigeracao = requerRefrigeracao;
    }

    public String getDescricao() { return this.descricao; }
    public boolean isRequerRefrigeracao() { return this.requerRefrigeracao; }
}
```